Lab Task 1: Student Records Management

Scenario:

You are building a **student record management system** for a university. Student records are stored by **Roll Number** and must always remain efficiently searchable.
To ensure consistent performance even as data grows, the system uses an **AVL Tree** instead of a plain BST to keep the records balanced after every insertion or deletion.

Data Model:

Each node represents a student record with:

- `int rollNo` (unique key)
- `string name`
- `float cgpa`

Core Functionalities:

- `addStudent(int rollNo, string name, float cgpa)`
- `findStudent(int rollNo)`
- `removeStudent(int rollNo)`
- `displayInorder()`

Advanced Operations:

- `getTopStudents(float threshold)` → returns all students with CGPA ≥ threshold
- `getTotalStudents()`

- `getHeight()`
- `isBalanced()` - verifies AVL property for all nodes

Test Cases

```
#include "gtest/gtest.h"
#include "StudentAVL.h"


// 1. Duplicate Roll Numbers Should Be Rejected
TEST(StudentAVLTest, RejectDuplicateRollNumbers) {
    StudentAVL avl;
    EXPECT_TRUE(avl.addStudent(101, "Alice", 3.5));
    EXPECT_FALSE(avl.addStudent(101, "Bob", 3.8)); // Duplicate roll
number
    EXPECT_EQ(avl.getTotalStudents(), 1);
}


// 2. Search for Non-Existing Roll Number Returns Empty Record
TEST(StudentAVLTest, FindNonExistingStudent) {
    StudentAVL avl;
    avl.addStudent(1, "A", 3.1);
    Student s = avl.findStudent(999);
    EXPECT_EQ(s.rollNo, -1);
    EXPECT_STREQ(s.name, "");
```

```
    EXPECT_FLOAT_EQ(s.cgpa, 0.0);
}

// 3. Left-Right Rotation Triggered
TEST(StudentAVLTest, LeftRightRotation) {
    StudentAVL avl;
    avl.addStudent(30, "A", 3.0);
    avl.addStudent(10, "B", 2.8);
    avl.addStudent(20, "C", 3.6); // triggers LR rotation
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 3);
}

// 4. Right-Left Rotation Triggered
TEST(StudentAVLTest, RightLeftRotation) {
    StudentAVL avl;
    avl.addStudent(10, "A", 3.0);
    avl.addStudent(30, "B", 3.2);
    avl.addStudent(20, "C", 3.6); // triggers RL rotation
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 3);
}
```

```
// 5. Deleting Root Node Rebalances Correctly

TEST(StudentAVLTest, DeleteRootNode) {

    StudentAVL avl;

    avl.addStudent(50, "X", 3.2);

    avl.addStudent(30, "Y", 3.4);

    avl.addStudent(70, "Z", 3.6);

    EXPECT_TRUE(avl.removeStudent(50)); // Delete root

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_EQ(avl.getTotalStudents(), 2);

}



// 6. Query Students Within Roll Number Range

TEST(StudentAVLTest, RangeQueryByRollNumber) {

    StudentAVL avl;

    avl.addStudent(10, "A", 2.5);

    avl.addStudent(20, "B", 3.5);

    avl.addStudent(30, "C", 3.0);

    avl.addStudent(40, "D", 3.8);



    int count = 0;

    Student* subset = avl.getRangeOfStudents(15, 35, count);

    EXPECT_EQ(count, 2);

    EXPECT_EQ(subset[0].rollNo, 20);
```

```
    EXPECT_EQ(subset[1].rollNo, 30);
    delete[] subset;
}

// 7. All Students with CGPA Above Threshold Returned
TEST(StudentAVLTest, HighAchieversQuery) {
    StudentAVL avl;
    avl.addStudent(1, "A", 3.1);
    avl.addStudent(2, "B", 3.9);
    avl.addStudent(3, "C", 2.7);
    avl.addStudent(4, "D", 3.8);

    int count = 0;
    Student* top = avl.getTopStudents(3.6, count);
    EXPECT_EQ(count, 2); // B and D
    delete[] top;
}

// 8. Tree Height Reflects Balance After Bulk Inserts
TEST(StudentAVLTest, HeightAfterMultipleInsertions) {
    StudentAVL avl;
    for (int i = 1; i <= 20; ++i)
        avl.addStudent(i, "X", 3.0f);
```

```
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 8);
}

// 9. Delete Non-Existing Student Should Fail Gracefully
TEST(StudentAVLTest, DeleteNonExistingStudent) {
    StudentAVL avl;
    avl.addStudent(10, "A", 3.0);
    avl.addStudent(20, "B", 3.2);
    EXPECT_FALSE(avl.removeStudent(999)); // Non-existing roll number
    EXPECT_EQ(avl.getTotalStudents(), 2);
    EXPECT_TRUE(avl.isBalanced());
}

// 10. Large Sequential Inserts and Deletes Preserve Balance
TEST(StudentAVLTest, LargeInsertDeleteSequence) {
    StudentAVL avl;
    const int N = 100;
    for (int i = 1; i <= N; ++i)
        avl.addStudent(i, "S", 3.0f + (i % 10) * 0.1f);
    for (int i = 1; i <= N; i += 2)
        avl.removeStudent(i);
    EXPECT_TRUE(avl.isBalanced());
```

```
    EXPECT_EQ(avl.getTotalStudents(), N / 2);
}


// 11. Verify Re-Insertion After Deletion Maintains Balance
TEST(StudentAVLTest, ReinsertAfterDelete) {
    StudentAVL avl;
    avl.addStudent(10, "A", 3.3);
    avl.addStudent(20, "B", 3.4);
    avl.addStudent(30, "C", 3.5);
    avl.removeStudent(20);
    EXPECT_TRUE(avl.isBalanced());
    avl.addStudent(20, "B2", 3.9);
    EXPECT_TRUE(avl.isBalanced());
    Student s = avl.findStudent(20);
    EXPECT_FLOAT_EQ(s.cgpa, 3.9);
}


// 12. Boundary Roll Numbers (Minimum and Maximum)
TEST(StudentAVLTest, BoundaryRollNumbers) {
    StudentAVL avl;
    avl.addStudent(1, "Lowest", 3.1);
    avl.addStudent(9999, "Highest", 3.8);
    EXPECT_TRUE(avl.isBalanced());
```

```
    Student s1 = avl.findStudent(1);
    Student s2 = avl.findStudent(9999);
    EXPECT_EQ(s1.rollNo, 1);
    EXPECT_EQ(s2.rollNo, 9999);
}

// 13. Check Balance After Sequential Descending Inserts
TEST(StudentAVLTest, DescendingInsertionsBalance) {
    StudentAVL avl;
    for (int i = 10; i >= 1; --i)
        avl.addStudent(i, "X", 3.0);
    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 6);
}

// 14. Delete All Students One by One
TEST(StudentAVLTest, DeleteAllStudents) {
    StudentAVL avl;
    avl.addStudent(10, "A", 3.1);
    avl.addStudent(20, "B", 3.2);
    avl.addStudent(30, "C", 3.3);
```

```
    avl.removeStudent(10);
    avl.removeStudent(20);
    avl.removeStudent(30);

    EXPECT_EQ(avl.getTotalStudents(), 0);
    EXPECT_TRUE(avl.isBalanced());
}

// 15. Verify Height and Count Are Updated Correctly
TEST(StudentAVLTest, CountAndHeightAfterOperations) {
    StudentAVL avl;
    avl.addStudent(1, "A", 3.5);
    avl.addStudent(2, "B", 3.6);
    avl.addStudent(3, "C", 3.7);
    EXPECT_EQ(avl.getTotalStudents(), 3);
    avl.removeStudent(2);
    EXPECT_EQ(avl.getTotalStudents(), 2);
    EXPECT_TRUE(avl.isBalanced());
}

// 16. Update (Replace) Student's CGPA by Removing & Reinserting
TEST(StudentAVLTest, UpdateStudentRecord) {
    StudentAVL avl;
```

```
    avl.addStudent(55, "Liam", 2.9);
    avl.removeStudent(55);
    avl.addStudent(55, "Liam", 3.6);
    Student s = avl.findStudent(55);
    EXPECT_FLOAT_EQ(s.cgpa, 3.6);
    EXPECT_TRUE(avl.isBalanced());
}

// 17. Minimum and Maximum Roll Retrieval via Traversal
TEST(StudentAVLTest, MinMaxRollTraversal) {
    StudentAVL avl;
    avl.addStudent(100, "A", 3.1);
    avl.addStudent(50, "B", 3.3);
    avl.addStudent(150, "C", 3.2);

    Student min = avl.findStudent(50);
    Student max = avl.findStudent(150);
    EXPECT_EQ(min.rollNo, 50);
    EXPECT_EQ(max.rollNo, 150);
}

// 18. CGPA Threshold Edge Case (Exactly Equal to Threshold)
TEST(StudentAVLTest, ThresholdEqualToCGPA) {
```

```
    StudentAVL avl;

    avl.addStudent(10, "A", 3.6);

    avl.addStudent(20, "B", 3.6);

    avl.addStudent(30, "C", 3.5);

    int count = 0;

    Student* arr = avl.getTopStudents(3.6, count);

    EXPECT_EQ(count, 2);

    delete[] arr;

}
```

Lab Task 2: Course Scheduling

Scenario:

Build an **AVL Tree** based course scheduling system where each course is keyed by its **startTime** (an `int` representing minutes since midnight). The AVL property must be preserved after every insertion and deletion. Beyond correctness of the tree, implement efficient domain-specific queries such as finding overlapping courses and computing free time slots.

The registrar needs a scheduler that stores course slots ordered by their start time and supports fast queries and updates. The system must:

- Keep course records balanced so queries remain `O(log n)`.
- Allow finding all courses that conflict with (overlap) a given time interval.
- Compute all free time intervals (gaps) between courses during a day.

- Support insertion and deletion of courses and always maintain AVL balance.
- Provide diagnostic functions to verify AVL-property and tree height.

Use an **AVL tree keyed by `startTime`**. If two courses have the same `startTime`, reject the insertion (start times must be unique for simplicity). Times are integers in minutes in range `[0, 1440)`; courses have `startTime < endTime`. The day runs from `0` (00:00) to `1440` (24:00).

Data Model and API

```
// Course record
struct Course {
    int courseID;      // unique course identifier
    int startTime;     // minutes since midnight [0, 1440)
    int endTime;       // minutes since midnight, startTime < endTime
    char title[128];   // fixed-size C-string for simplicity
};

// AVL node
struct Node {
    Course data;
    Node* left;
    Node* right;
    int height;
};

// Scheduler API (member functions of CourseAVL class)
class CourseAVL {
public:
    CourseAVL();
    ~CourseAVL();

    // Core operations
    bool addCourse(int courseID, int startTime, int endTime, const char*
title);
    bool removeCourseByStart(int startTime);      // remove by key
    Course* findCourseByStart(int startTime);    // returns pointer or
nullptr
```

```
    // Queries
    // findOverlaps returns dynamic array (allocated with new[]) and sets
outCount.
    // Caller is responsible for deleting the returned array with
delete[].
    Course* findOverlaps(int qStart, int qEnd, int &outCount);

    // getFreeSlots returns an array of pairs (start, end) as Course-like
objects
    // where courseID = -1 and title empty; outCount set to number of
gaps.
    Course* getFreeSlots(int &outCount);

    // Diagnostics
    int getTotalCourses();
    int getHeight();
    bool isBalanced(); // verifies AVL property across all nodes

    // Utility: pretty print (inorder) - prints to stdout
    void displayInorder();

    // (internal helpers are allowed)
};
```

Requirements & Constraints

1. **AVL correctness**
    - After every `addCourse` and `removeCourseByStart`, the tree must be height-balanced (balance factor ∈ {-1,0,1} for every node).
    - Rotations (LL, RR, LR, RL) must update `height` correctly.
2. **Key uniqueness**
    - `startTime` is the key. If `addCourse` receives a `startTime` that already exists, it should reject the insert and return `false`.

3. **Validation**
    - Reject invalid times (e.g., `startTime >= endTime`, or times outside `[0,1440]`).
4. **Overlap definition**
    - Two intervals `[a,b)` and `[c,d)` overlap if `max(a,c) < min(b,d)`.
    - Example: `[9:00,10:00)` and `[9:30,10:30)` overlap.
5. **Free slot computation**
    - The day is `[0, 1440)`. Free slots are maximal intervals not covered by any course.
    - Adjacent courses (endTime == next startTime) produce no gap between them.
    - If there are no courses, a single free slot `[0,1440)` is returned.
6. **Memory**
    - No STL containers. Dynamic arrays can be returned via `new` with size communicated via `outCount`.
    - Ensure no memory leaks (delete nodes in destructor).
7. **Efficiency targets**
    - `addCourse`, `removeCourseByStart`, `findCourseByStart`: `O(log n)` average/worst (due to AVL).
    - `findOverlaps(qStart, qEnd)`: should be `O(k + log n)` where `k` is number of overlapping courses (use interval pruning).
    - `getFreeSlots`: `O(n)` (inorder traversal once).

Implementation Notes:

1. Store courses keyed by `startTime`.
2. For `findOverlaps`, perform a modified inorder or recursive search:

- If a node's `startTime` >= `qEnd`, only search left subtree.
- If a node's `endTime` <= `qStart`, only search right subtree.
- Otherwise, the node overlaps; record it and search both subtrees.

3. For `getFreeSlots`:
   - Traverse nodes in inorder; track `previousEnd` starting at 0.
   - For each node visited with interval `[s,e)`, if `s > previousEnd` add gap `[previousEnd, s)`, then set `previousEnd = max(previousEnd, e)`.
   - After traversal, if `previousEnd < 1440`, add final gap `[previousEnd, 1440)`.
4. For `isBalanced`, compute heights bottom-up and verify balance factor for each node (return false on any violation).
   - Carefully update `height` in rotations and in insert/delete rebalancing code.
5. For deletion of a node with two children, replace with inorder successor (min of right subtree), then rebalance upwards.

Test Cases

```
TEST(CourseSchedulerTest, InsertAndFind) {
    CourseAVL sched;
    bool ok;

    ok = sched.addCourse(101,  540,  600, "CS101"); // 9:00-10:00
    EXPECT_TRUE(ok);
    ok = sched.addCourse(102,  600,  660, "CS102"); // 10:00-11:00
    EXPECT_TRUE(ok);

    Course* c = sched.findCourseByStart(540);
    ASSERT_TRUE(c != nullptr);
    EXPECT_EQ(c->courseID, 101);
    EXPECT_EQ(c->startTime, 540);
    EXPECT_EQ(c->endTime, 600);
```

```
    EXPECT_EQ(sched.getTotalCourses(), 2);
    EXPECT_TRUE(sched.isBalanced());
}
TEST(CourseSchedulerTest, RejectDuplicatesAndInvalid) {
    CourseAVL sched;
    bool ok;
    ok = sched.addCourse(201, 480, 540, "Math"); // valid
    EXPECT_TRUE(ok);
    ok = sched.addCourse(202, 480, 520, "Physics"); // duplicate start
    EXPECT_FALSE(ok); // rejected

    ok = sched.addCourse(203, 700, 700, "ZeroLength"); // invalid
(start==end)
    EXPECT_FALSE(ok);

    ok = sched.addCourse(204, -10, 50, "NegativeStart"); // invalid
    EXPECT_FALSE(ok);
}
TEST(CourseSchedulerTest, FindOverlaps) {
    CourseAVL sched;
    sched.addCourse(1,  540, 600, "A"); // 9:00-10:00
    sched.addCourse(2,  570, 630, "B"); // 9:30-10:30
    sched.addCourse(3,  660, 720, "C"); // 11:00-12:00
    sched.addCourse(4,  600, 660, "D"); // 10:00-11:00

    int count = 0;
    Course* overlaps = sched.findOverlaps(580, 615, count); // 9:40-10:15

    // Expected overlapping courses: 1 (9:00-10:00), 2 (9:30-10:30), 4
(10:00-11:00) -> 3 items
    EXPECT_EQ(count, 3);

    // Check that returned items are indeed overlapping and have proper
times
    for (int i = 0; i < count; ++i) {
        EXPECT_TRUE(max(overlaps[i].startTime, 580) <
min(overlaps[i].endTime, 615));
    }
    delete[] overlaps;
}
```

```
TEST(CourseSchedulerTest, GetFreeSlots) {
    CourseAVL sched;
    // Day: 0-1440
    sched.addCourse(10,  480, 540, "Morning");   // 8:00-9:00
    sched.addCourse(11,  600, 660, "LateMorning"); // 10:00-11:00
    sched.addCourse(12,  720, 780, "Noon");      // 12:00-13:00

    int count = 0;
    Course* gaps = sched.getFreeSlots(count);

    // Gaps expected:
    // [0,480), [540,600), [660,720), [780,1440)  --> 4 gaps
    EXPECT_EQ(count, 4);

    // Check each gap is valid (courseID == -1 indicates gap)
    EXPECT_EQ(gaps[0].courseID, -1);
    EXPECT_EQ(gaps[0].startTime, 0);
    EXPECT_EQ(gaps[0].endTime, 480);

    EXPECT_EQ(gaps[3].startTime, 780);
    EXPECT_EQ(gaps[3].endTime, 1440);

    delete[] gaps;
}


TEST(CourseSchedulerTest, DeletionCausesRotation) {
    CourseAVL sched;
    // Insert in order that will create skew and require rotations when
deleting
    sched.addCourse(1,  300, 330, "A");
    sched.addCourse(2,  200, 230, "B");
    sched.addCourse(3,  100, 130, "C"); // causes imbalance (left-left)
    EXPECT_TRUE(sched.isBalanced());

    // Remove the left-most to force rebalancing on path
    bool ok = sched.removeCourseByStart(100);
    EXPECT_TRUE(ok);
    EXPECT_TRUE(sched.isBalanced());
    EXPECT_EQ(sched.getTotalCourses(), 2);
```

```
}
TEST(CourseSchedulerTest, FullDayAndBackToBack) {
    CourseAVL sched;
    // Full day course
    bool ok = sched.addCourse(50, 0, 1440, "FullDay");
    EXPECT_TRUE(ok);

    int count = 0;
    Course* gaps = sched.getFreeSlots(count);
    // No gaps expected
    EXPECT_EQ(count, 0);
    delete[] gaps;

    // Remove full day
    EXPECT_TRUE(sched.removeCourseByStart(0)); // removes startTime == 0
node

    // Back-to-back courses
    sched.addCourse(60, 480, 540, "A"); // 8:00-9:00
    sched.addCourse(61, 540, 600, "B"); // 9:00-10:00 (back-to-back)
    count = 0;
    gaps = sched.getFreeSlots(count);
    // Expect gaps: [0,480), [600,1440) -> 2 gaps
    EXPECT_EQ(count, 2);
    delete[] gaps;
}
```